

Reviewer Pack — Minimal Local Induction Ring Rank and Communication Complexi...

Entry 0e5b65fdaa8d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 01:47:56 UTC

Minimal Local Induction Ring Rank and Communication Complexity Rank Variance

Entry ID: 0e5b65fdaa8d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 01:47:56 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Local Induction Rings) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every CNF φ with n variables, the variance of the communication complexity rank of its associated matrix $A(\varphi)$ is upper-bounded by a constant multiple of the local induction ring rank of the field K associated with φ , i.e., $\text{Var}(\text{rank}(A(\varphi))) \leq c \cdot \text{LIR}(K)$, where c is a constant.

Rationale (proposer's reasoning):

Local induction rings (LIRs) provide a framework for studying the structure of algebraic objects. By linking the LIR rank to communication complexity rank variance, we might uncover new insights into the computational hardness of SAT and related problems.

Taxonomy category: CommunicationComplexityMatrixRank (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0e9aa18fa5d577f6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of the generated CNFs with up to 40 variables, the variance of the communication complexity rank of $A(\varphi)$ is less than or equal to a predefined constant c times the local induction ring rank $\text{LIR}(K)$ associated with each CNF. The criterion is falsified if any seed produces a variance greater than $c \cdot \text{LIR}(K)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=248.7s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n - 1):
            clause = [random.choice([f'x{i+1}', f'-x{i+1}']) for i in range(n)]
            clauses.append(clause)
        return clauses

    def matrix_from_cnf(cnf, n):
        m = len(cnf)
        A = [[0] * (n + 1) for _ in range(m)]
        for i, clause in enumerate(cnf):
            for literal in clause:
                if literal[0] == '-':
                    var = int(literal[1:]) - 1
                    A[i][var] = -1
                else:
                    var = int(literal[1:]) - 1
                    A[i][var] = 1
        return A

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            if A[i][i] == 0:
                for j in range(i + 1, m):
                    if A[j][i] != 0:
                        A[i], A[j] = A[j], A[i]
                        break
            if A[i][i] == 0:
                continue
```

```

        pivot = Fraction(A[i][i])
        for j in range(n):
            A[i][j] /= pivot
        for j in range(m):
            if j != i and A[j][i] != 0:
                factor = -A[j][i]
                for k in range(n):
                    A[j][k] += factor * A[i][k]
    rank = sum(1 for row in A if any(row))
    return rank

def local_induction_ring_rank(K):
    # Placeholder for actual LIR calculation
    # This is a dummy implementation and should be replaced with the correct method
    return 1 # Example value, replace with actual LIR calculation

n = random.randint(5, 40)
cnf = generate_cnf(n)
A = matrix_from_cnf(cnf, n)
rank_A = gaussian_elimination(A)
K = local_induction_ring_rank("Q") # Placeholder for the field K
c = Fraction(1, 1) # Example constant, replace with actual calculation

variance = 0
instances_tested = 30
n_max = n

for _ in range(instances_tested):
    rank_A_instance = gaussian_elimination(A)
    variance += (rank_A_instance - rank_A) ** 2

variance /= instances_tested
conjecture_holds = variance <= c * K
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Variance of Communication Complexity Rank",
    "metric_value": float(variance),
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_dev = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))

```

```

support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
else:
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not meet the pre-registered support condition for a clear SUPPORTED verdict. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13912
2	preregistration	ollama_remote	glm4:latest	0	0	9817
3	novelty	ollama_remote	glm4:latest	0	0	9298
4	novelty	ollama_remote	glm4:latest	0	0	16127
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16126
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12341
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11810
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11392
9	judge	ollama_remote	glm4:latest	0	0	80762

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 181585 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in <research/audit/0e5b65fdaa8d.jsonl>)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0e5b65fdaa8d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0e5b65fdaa8d.tar.gz` (if generated)